商品搜索：从 LIKE 到 ES + Milvus hybrid

用代码 + 业务双视角讲清 gomall 的搜索全链路

gomall · deck 03 (v2)

今日大纲

搜索是流量入口，不是普通功能

用户进站后第一件事就是搜

- 进站用户里约 60% 会主动用搜索，是漏斗最宽的一跳。
- ” 搜不到 / 搜偏” 用户不会换词，直接退出 App。
- 入口处的相关性，决定后面三跳还能不能继续。

routes/router.go:41 /product/search 是入口；:42 /product/semantic-search 是 hybrid 入口

业务对搜索的四条硬约定

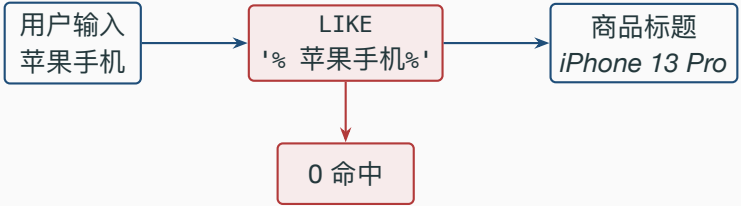
- **召回**：用户搜得到的词，能跑出至少一条相关结果。
- **时延**：p95 < 200ms，列表页才不会”卡半秒空白”。
- **新鲜度**：商家上架 → 站内可搜，SLA ≤ 5min。
- **兜底**：ES 挂掉，搜索接口不能 5xx，必须有降级路径。

> 这四条决定了搜索系统不能只是 `SELECT * WHERE name LIKE '%kw%'`。

stressTest/REPORT.md：/product/show 同库 PK 查询基线 62,226 RPS / p95 3ms 作为时延锚点

先看反面教材：LIKE 为什么不够

用户搜” 苹果手机”，DB 搜不到 iPhone 13



- 用户脑里是中文，商家上架习惯写英文型号。
- LIKE 不知道” 苹果 == Apple == iPhone”，子串不匹配 = 0 命中。
- 这不是用户拼错，是搜索系统欠用户一个” 分词 + 同义” 能力。

repository/db/dao/product.go:80–98 是当前 DB 兜底用的 LIKE 实现

LIKE 的第二个问题：跑不动

指标	数值
product 表行数	956K 行
product 表磁盘大小	364 MB
LIKE `'%X%'` 能走索引?	否（前缀通配）
全表扫预估单 query 耗时	秒级

- `LIKE '%kw%'` 前缀是通配符，B+ 树用不上，必扫全表。
- 96 万行 + 364 MB 数据，单查就吃满一颗 CPU。
- 对照：同库 `/product/show` 走主键命中 **62,226 RPS / p95 3ms**。

stressTest/REPORT.md § 数据规模 + 链路压测表第 1 行

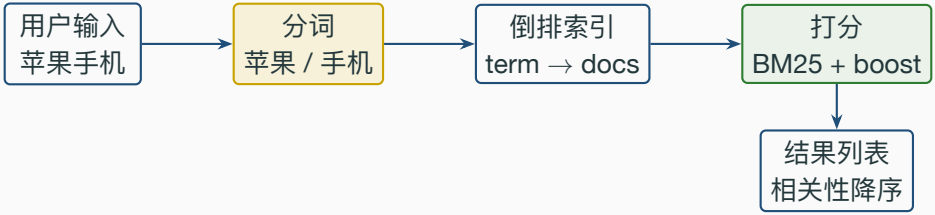
LIKE 缺的三件事，全是业务能力

能力	LIKE 有?	业务后果
分词	无	” 苹果手机” 搜不到”iPhone”
字段加权	无	标题命中 vs 详情命中等权排
高亮	无	列表里看不到为什么命中

- 分词决定召回，字段加权决定排序，高亮决定点击率。
- 三件事任一缺失，” 搜索” 就退化成” 碰运气包含”。
- 这才是 gomall 换 ES 的业务动机，不是图组件新潮。

ES 关键词层：分词 / 加权 / 高亮

ES 替 LIKE 做什么



- **分词**：按词切（中文走 IK 或 standard fallback）。
- **倒排索引**：term 直接索引到 doc 列表，避开全表扫。
- **打分**：`multi_match` + 字段加权，name > title > info。

repository/es/product_index.go:163–227 SearchProductsWithScore 多字段加权查询

字段加权背后的业务判断

字段	业务含义	加权
`name`	商品主名（用户预期匹配）	$\times 3$
`title`	商品标题（含型号 / 规格）	$\times 2$
`info`	商品详情（长文，噪声多）	$\times 1$

- 用户搜”iPhone”，” 标题就叫 iPhone 13” 应当排在”info 提到 iPhone” 前面。
- 加权由运营调，不是开发拍。大促可临时把 `title` 上调到 $\times 4$。
- 不加权 $\rightarrow$ ” 标签店家把热词塞进 info” 会霸占第一屏。

repository/es/product_index.go:174–177 `fields={name^3,title^2,info}`

代码段 1：ES 关键词检索（含 _score 输出）

```go
170 must := []map[string]any{{
171    "multi_match": map[string]any{
172       "query": keyword,
173       "fields": []string{"name^3", "title^2", "info"},
174    },
175 }}
176 filter := []map[string]any{}
177 if categoryID != nil {
178    filter = append(filter, map[string]any{
179       "term": map[string]any{"category_id": *categoryID},
180    })
181 }
182 q := map[string]any{
183    "from": from, "size": size,
184    "query": map[string]any{"bool": map[string]any{
185       "must": must, "filter": filter,
186    }},
187 }
```

repository/es/product_index.go:170–191 关键片段

代码段 1 逐行讲解

- **L170–175**：`multi_match` 同时查 name / title / info，靠 `^N` 给字段加权。这一行就是上一页那张加权表的代码落点。
- **L176–180**：`filter` 走 `term`，类目过滤不参与打分，只做 yes/no 过滤，性能更稳。运营按类目限定时进这条分支。
- **L182–189**：`bool.must + filter` 是 ES 的标准组合——must 决定相关性，filter 决定可见性。两者分离，分页和打分才不会互相干扰。
- **业务对应**：name 权重最高，对应”用户最在意主名命中”；filter 不打分，对应”类目是硬过滤，不是建议”。

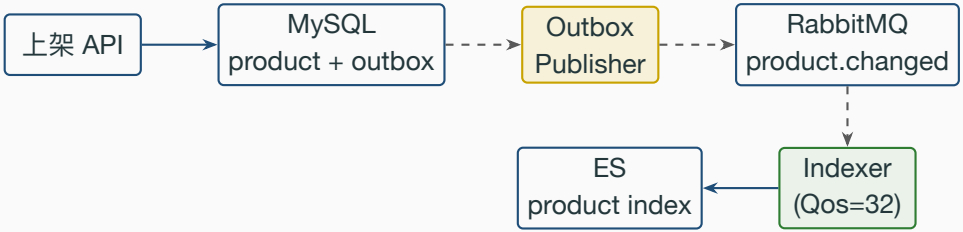
上架到可搜：outbox + 异步索引

业务承诺：商家上架 5 分钟内必须搜得到

- 商家后台点”上架” → 站内搜索能搜到，业务侧的 SLA 是 $\leq$ 5min。
- 5min 是商家容忍上限：拍照 / 调价 / 改文案的反馈节奏。
- 写 DB 同步写 ES 行不行？**不行**：ES 抖动直接 fail 上架，商家退出。
- 必须**解耦**：DB 写完就返回成功，ES 异步追上。

service/product.go:155, 253, 289 三处商品写入后调 emitProductChanged

outbox → MQ → indexer 的链路



- 上架事务里只写 product + outbox，二者同库同事务，原子。
- Publisher 后台扫 outbox → 投递到 `product.changed` 路由。
- Indexer 消费后写 ES。Qos=32 控制并发，避免 ES 被写穿。

service/product.go:257–265 emitProductChanged；service/search/indexer.go:14–49 indexer 主循环

代码段 2：indexer 主循环

```go
17 func StartProductIndexer(ctx context.Context) error {
18    rabbitmq.BindDomainQueue(indexerQueue, "product.changed")
19    ch, _ := rabbitmq.GlobalRabbitMQ.Channel()
20    ch.Qos(32, 0, false) // 在途上限 32 条
21    msgs, _ := ch.Consume(indexerQueue, "", false, false, false, false, nil)
22    go func() {
23       for d := range msgs {
24          var ev events.ProductChanged
25          if json.Unmarshal(d.Body, &ev) != nil {
26             d.Nack(false, false); continue // 脏消息进死信
27          }
28          if handleProductChanged(ctx, ev) != nil {
29             d.Nack(false, true); continue // 重投自愈
30          }
31          d.Ack(false)
32       }
33    }()
34    return nil
35 }
```

service/search/indexer.go:17–49（折行省略错误处理）

代码段 2 逐行讲解

- **L18–19**：`BindDomainQueue` 把 `product.changed` 路由绑到 `search.product.indexer` 队列。上架 / 改价 / 下架共用一条路由。
- **L22**：`Qos(32, 0, false)` 让消费端最多在途 32 条未 ack。这是给 ES 写入设的”流量阀”，对应 deck 05 的”在网关之外，每个消费者也要节流”。
- **L30–34**：JSON 解析失败 → `Nack(false, false)`，不重投。脏消息进死信，不阻塞健康消息。
- **L35–38**：业务处理失败 → `Nack(false, true)` 重投，给 ES 短抖动一次自愈机会。
- **L39**：成功才 ack，保证 at-least-once。索引是幂等的（同 docID 覆盖），重复消费无害。

service/search/indexer.go:25 Qos=32；:36/:42 Nack 语义

对应 ES 写入：UpsertProduct 是幂等的

- `op=create/update` → `UpsertProduct(p)`；`op=delete` → `DeleteProduct(id)`。
- `DocumentID = product.ID`：同 ID 多次写入覆盖，天然幂等，对应消费侧的”重投无害”。
- `Refresh: "false"`：不强制刷盘，让 ES 自己按 1s 刷新窗口批量落盘，吞吐优先。
- 5min SLA 实际由 RMQ 堆积长度兜底：Qos=32 下，95% 场景 < 30s 可搜。

service/search/indexer.go:51–60 handleProductChanged；repository/es/product_index.go:105–131 UpsertProduct

代码段 3：product mapping（建索引时的 schema）

```
20 "settings": { "number_of_shards": 1, "number_of_replicas": 0 },
21 "mappings": { "properties": {
22   "name": { "type": "text", "analyzer": "standard" },
23   "title": { "type": "text", "analyzer": "standard" },
24   "info": { "type": "text", "analyzer": "standard" },
25   "category_id": { "type": "long" },
26   "price": { "type": "keyword" },
27   "on_sale": { "type": "boolean" },
28   "created_at": { "type": "date", "format": "epoch_second" }
29 }}
```

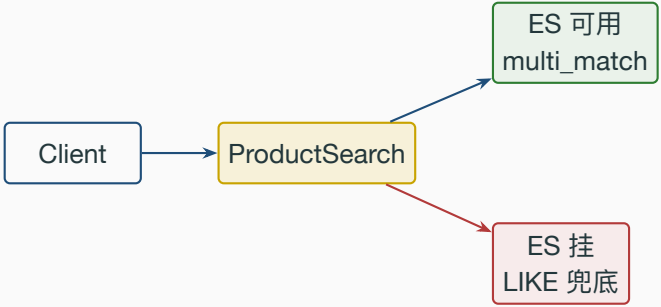
repository/es/product_index.go:20–41 productIndexMapping（节选）

代码段 3 逐行讲解

- **shards=1 / replicas=0**：gomall 单节点部署，不做多副本。生产换多节点要把 replicas 拉到 1，read 走主 + 副两副本。
- **name/title/info 是 `text`**：参与分词 + 倒排，是关键词召回的目标字段。
- **price 是 `keyword`**：不分词，整串做 filter / exact match。价格永远不会被” 分词” 成”99” + ” 元”。
- **category_id / boss_id 是 `long`**：做 `term` 过滤（按类目 / 按店铺筛），不打分。
- **on_sale 是 `boolean`**：下架商品要从结果里隐藏，靠这个字段做硬过滤。
- **analyzer=standard**：未装 IK 分词器时的 fallback，保证集群裸装也能跑通。

ES 挂了：静默降级到 DB

业务约定：搜索接口不许 5xx，启动也不许阻塞



- 5xx 等于 app 崩页，比” 结果不准” 业务损失大得多。LIKE 慢也能用。
- `tryInitES` 用 `defer recover()` 把启动 panic 压平成 warn ——ES 故障不阻塞启动。
- **ES 是优化项，不是依赖项。**

service/product.go:294–323 兜底；cmd/main.go:64–73 tryInitES；initialize/search.go:13–31

语义层：ES + Milvus hybrid

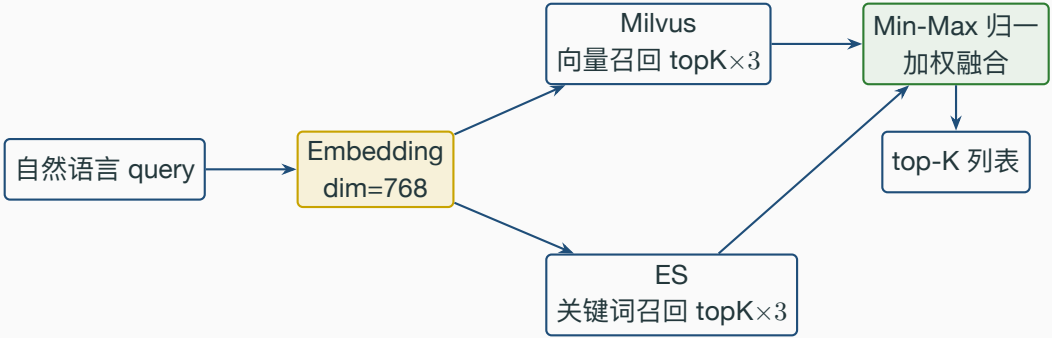
ES 关键词搞不定的三类 query

query 类型	例子	ES 行为
自然语言	送女朋友的礼物	拆成” 送/女朋友/礼物” → 命中乱
跨表达同义	苹果手机最新款	名义命中 iPhone 15 漏
口语描述	适合健身的运动鞋	长尾词命中率趋近 0

- ES 衡量**词重合度**，不是**语义接近度**。
- 用户被短视频 / LLM 养成自然语言搜的习惯，长尾流量比例上升。
- 长尾召回率掉到 60% 以下，整段 GMV 静默漏走，没有告警。

service/search/semantic.go:1–56 hybrid 入口注释与默认权重

hybrid 三层：关键词 + 向量 + 业务调权



向量管召回，关键词管精度，融合权重管业务偏好。*service/search/semantic.go:54–143*

semanticSearchWith 主流程

融合权重：业务调权的入口

权重 (sem : kw)	行为	适合的 query
80 : 20	偏语义	” 送女朋友的礼物” 自然语言
50 : 50	中庸	gomall 默认
20 : 80	偏关键词	”iPhone 13 256G” 品牌型号
0 : 100	退化为 ES	ES 全场景兜底

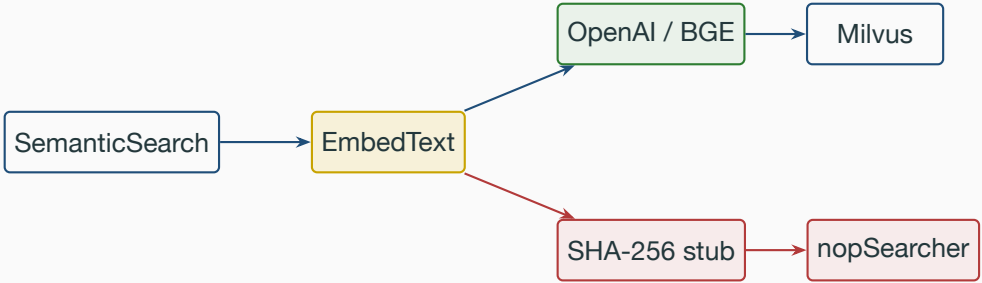
- 50/50 是无偏起点，留 A/B 空间；短 query 偏关键词，长 query 偏语义。
- 大促” 今日特卖” 强插不改算法：**在融合分外面加业务分**

```
Score = wSem*sem + wKw*kw + promote(id)
```

- 不污染长期相关性，运营随时下线。

service/search/semantic.go:21–23 权重常量；*:107–111 Score* 处可挂业务 *boost*

Embedding 与 Milvus 都做了可降级



- `EMBEDDING_API_URL` 未配 → SHA-256 stub 兜底；上线节奏 OpenAI → BGE 省 token；dim=768 硬约束。
- Milvus 走 `MilvusSearcher` 接口，未注入 → nop 返回 nil → hybrid 退化为纯 ES。
- **Milvus 挂只丢语义，关键词召回不受影响。**

service/search/embedding.go:52–113；service/search/milvus_stub.go:9–47；cmd/main.go:86–97 tryInitMilvus

业务侧总结

一张表把这套搜索系统讲完

能力	实现	代码位置
关键词召回	ES `multi_match`	repository/es/product_index.go
增量索引	outbox → MQ → indexer	service/search/indexer.go
DB 兜底	ES 客户端为 nil → LIKE	service/product.go:294
全量回填	admin 触发的 Backfill	service/search/backfill.go
语义召回	Milvus + embedding	service/search/semantic.go
模型切换	env: `EMBEDDING_API_URL`	service/search/embedding.go
启动静默	defer recover	cmd/main.go:64–73

routes/router.go:41–42 search + semantic-search 两个入口；:154 admin/search/backfill

四方视角下的搜索系统

- **用户**：搜什么都能搜到，第一屏都是想买的。
- **商家**：上架 5 分钟内被收录，下架后立刻消失。
- **运营**：按类目 / 大促 / 特卖手动调权，不用 PR 工程师。
- **SRE**：ES 挂不告警业务，Milvus 挂只丢语义不丢搜索。

搜索副本系统的判据和详情页缓存一致：**副本旧一点点不要命，副本错乱才要命**。outbox 把” 上架成功” 和” 可搜” 解耦，是 5min SLA 的工程根据。

面试 Q&A

Q1：为什么不让上架 API 直接写 ES，而要 outbox?

A：DB 与 ES 不同事务，同步写要么失败回滚（伤体验），要么不一致（伤召回）。outbox 把”DB 写成功” 和”ES 写成功” 两件事拆开，分别保证。

Q2：ES 挂掉，搜索接口为什么不直接 5xx?

A：搜索是入口，5xx 等于 app 崩页。LIKE 慢一点也比白屏强——业务承诺” 搜索框永远可用”。

Q3：hybrid 50/50 这个权重你怎么调?

A：业务值，不是技术值。先 50/50 跑一轮，A/B 观察 CTR / 转化。短 query 偏关键词，长 query 偏语义。

代码位置 + 配套材料

搜索 / 语义入口 *routes/router.go:41–42*

ES 检索 + mapping *repository/es/product_index.go:20–41, 163–227*

outbox 事件 *service/product.go:257–265*

indexer 消费 *service/search/indexer.go:17–49*

ES 兜底分支 *service/product.go:294–323*

全量回填 *service/search/backfill.go:13–43*

hybrid 融合 + embedding *service/search/semantic.go, embedding.go*

Milvus 接口 *service/search/milvus_stub.go:9–47*

启动静默 *cmd/main.go:64–73 + initialize/search.go:13–31*

配套：deck 04 outbox / saga；deck 12 搜索业务漏斗 (v1)；deck 21 AI 检索；stressTest/REPORT.md。